**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ**

**ІМЕНІ ІВАНА ФРАНКА**

# ЗВІТ

# ДО ЛАБОРАТОРНОЇ РОБОТИ №8

«Мережеве та асинхронне програмування. REST API»

з курсу «Професійний Python»

Варіант №3 — «Інтернет-магазин»

Виконав(ла): Вересюк Ірина

Група: ФЕП-32

Перевірив(ла): Куньо І.М.

Дата: 01.11.26

Львів — 2026

# 1. Мета роботи

Ознайомитися з мережевим програмуванням у Python, протоколом HTTP, архітектурним стилем REST та форматом JSON. Навчитися створювати REST API за допомогою FastAPI, перевіряти дані через Pydantic, використовувати асинхронні операції async/await, asyncio.Task, asyncio.gather(), timeout, retry, Semaphore та HTTPX. Реалізувати API інтернет-магазину з persistence layer і автоматизованими тестами.

# 2. Індивідуальний варіант і постановка задачі

Варіант №3 — «Інтернет-магазин». Основні ресурси: products і categories. Необхідно реалізувати створення, перегляд, оновлення та видалення товарів; пошук і фільтрацію за категорією; покупку зі зменшенням залишку; визначення найдорожчого товару та загальної вартості залишків. Асинхронне завдання має демонструвати паралельне отримання кількох товарів з використанням Task, gather, Semaphore, HTTPX, timeout і retry.

# 3. Структура проєкту

```
InternetShop_Lab08_V3/
■■■ pyproject.toml
■■■ README.md
■■■ .gitignore
■■■ src/
■   ■■■ internet_shop/
■       ■■■ __init__.py
■       ■■■ api.py
■       ■■■ database.py
■       ■■■ models.py
■       ■■■ repositories.py
■       ■■■ schemas.py
■       ■■■ services.py
■       ■■■ external_api.py
■■■ tests/
■   ■■■ conftest.py
■   ■■■ test_api.py
■■■ report/
    ■■■ ■■■■■■■*■■■■■*■■■08_V3.pdf
```

Каталог src/internet_shop містить застосунок. api.py описує REST endpoints, schemas.py — Pydantic-моделі, models.py — ORM-моделі, repositories.py — доступ до даних, services.py — бізнес-логіку, database.py — підключення до БД, external_api.py — асинхронні HTTP-запити. Тести розміщено в tests/.

# 4. Опис реалізації

## 4.1. Моделі та persistence layer

Для збереження даних використано SQLAlchemy 2 і SQLite. Таблиця categories містить ідентифікатор та назву категорії. Таблиця products містить назву, опис, ціну, кількість на складі та зовнішній ключ category_id. База shop.db створюється автоматично під час запуску API.

## 4.2. Pydantic validation

Схеми ProductCreate і ProductPatch перевіряють вхідні дані. Назва повинна містити від 2 до 120 символів, ціна має бути більшою за нуль, кількість не може бути від’ємною, а category_id повинен бути додатним. Некоректний запит повертає HTTP 422.

## 4.3. REST endpoints

```
GET    /products
GET    /products/{product_id}
POST   /products
PATCH  /products/{product_id}
DELETE /products/{product_id}
GET    /categories/{category_id}/products
POST   /products/{product_id}/purchase
GET    /products/statistics/summary
POST   /products/batch
```

API використовує GET для читання, POST для створення та покупки, PATCH для часткового оновлення і DELETE для видалення. Path parameters передають ідентифікатори, query parameters — фільтри category_id та search. Дані передаються у форматі JSON.

### 4.4. Транзакційна покупка

Функція purchase_product перевіряє наявність товару та достатню кількість на складі. Якщо товару не існує, повертається 404; якщо залишку недостатньо — 409. Успішна покупка зменшує кількість і фіксує зміни через commit. У разі помилки виконується rollback.

### 4.5. Асинхронні HTTP-запити

У external_api.py реалізовано отримання товарів через httpx.AsyncClient. asyncio.create_task створює задачі, asyncio.gather очікує їх спільне завершення, asyncio.Semaphore(5) обмежує кількість одночасних запитів. Для мережевих запитів установлено timeout, а повторні спроби з паузою виконуються у разі тимчасової помилки. Endpoint POST /products/batch демонструє цю функціональність.

## 5. Налаштування pyproject.toml

Файл містить метадані проєкту, залежності FastAPI, Uvicorn, SQLAlchemy, Pydantic і HTTPX, а також додаткові залежності pytest та pytest-asyncio для тестування. Проєкт вимагає Python 3.11 або новішої версії.

## 6. Встановлення та запуск

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -e ".[test]"
uvicorn internet_shop.api:app --reload
```

Після запуску інтерактивна документація доступна за адресою http://127.0.0.1:8000/docs, а альтернативна документація — за адресою http://127.0.0.1:8000/redoc.

## 7. Приклади запитів

### 7.1. Створення категорії

```
POST /categories
{
  "name": "Electronics"
}
```

### 7.2. Створення товару

```
POST /products
{
  "name": "Smartphone",
  "description": "Test product",
  "price": 500.0,
  "quantity": 5,
  "category_id": 1
}
```

### 7.3. Купівля товару

```
POST /products/1/purchase
{
  "quantity": 2
}
```

У разі успішної покупки відповідь містить ідентифікатор товару, кількість придбаних одиниць, ціну за одиницю, суму покупки та залишок на складі.

## 8. Автоматизовані тести

У tests/test_api.py передбачено перевірки health-check endpoint, створення та отримання товару, валідації від’ємної ціни, часткового оновлення, видалення, успішної покупки, відмови при

недостатньому залишку, пошуку та статистики. Тести використовують окрему тимчасову SQLite-базу, щоб не змінювати робочі дані.

```
python -m pytest -v
```

Очікуваний результат за умови встановлених залежностей: усі вісім тестів завершуються успішно. Фактичний результат слід підтвердити запуском тестів у середовищі виконання.

## 9. HTTP status codes

200 OK — успішне отримання або оновлення; 201 Created — створення ресурсу; 204 No Content — успішне видалення; 404 Not Found — товар або категорію не знайдено; 409 Conflict — дублікат категорії або недостатній залишок; 422 Unprocessable Entity — помилка перевірки вхідних даних; 502/504 — помилка або перевищення часу асинхронного запиту.

## 10. Відповіді на контрольні запитання

1. Що таке HTTP? Протокол прикладного рівня для обміну запитами та відповідями між клієнтом і сервером.

2. Що містить HTTP-запит? Метод, URL, заголовки та, за потреби, тіло запиту.

3. Для чого використовують GET? Для отримання ресурсу або колекції ресурсів без зміни їхнього стану.

4. Для чого використовують POST? Для створення ресурсу або запуску операції, наприклад покупки товару.

5. Чим PATCH відрізняється від PUT? PATCH змінює лише передані поля, тоді як PUT зазвичай замінює ресурс повністю.

6. Що означає HTTP 404? Запитаний ресурс не знайдено.

7. Що таке REST API? API, організований навколо ресурсів і стандартних HTTP-методів, із передаванням представлень ресурсів.

8. Для чого потрібний JSON? Для компактного текстового подання структурованих даних.

9. Для чого використовується FastAPI? Для створення HTTP API з підтримкою типізації, валідації та автоматичної документації.

10. Яку роль виконує Pydantic? Перевіряє, перетворює та серіалізує дані відповідно до оголошених схем.

11. Що таке path parameter? Значення, що є частиною шляху, наприклад product_id у /products/{product_id}.

12. Що таке query parameter? Параметр після знака ?, який застосовують для фільтрації або налаштування запиту.

13. Що означають async та await? async оголошує coroutine, а await очікує завершення асинхронної операції.

14. Для чого потрібний asyncio.Task? Для планування coroutine на виконання в циклі подій.

15. Для чого потрібний asyncio.gather()? Для одночасного очікування кількох асинхронних операцій.

16. Для чого потрібний timeout? Щоб обмежити час очікування операції та уникнути необмеженого зависання.

17. Що таке retry? Повторне виконання запиту після тимчасової помилки.

18. Для чого потрібний asyncio.Semaphore? Для обмеження кількості операцій, які виконуються одночасно.

19. Що таке httpx.AsyncClient? Асинхронний HTTP-клієнт для надсилання запитів без блокування потоку на час очікування мережі.

20. Для чого потрібний Repository Layer? Для відокремлення операцій доступу до даних від бізнес-логіки та API.

21. Що таке транзакція? Група операцій із БД, які фіксуються разом або скасовуються у разі помилки.

22. Для чого потрібні автоматизовані API-тести? Щоб перевіряти endpoints, статуси відповідей, валідацію та бізнес-правила без ручного повторення запитів.

## 11. Висновок

У межах лабораторної роботи підготовлено проєкт REST API для інтернет-магазину за варіантом №3. Реалізовано роботу з категоріями та товарами, CRUD-операції, пошук і фільтрацію, покупку зі зменшенням залишку, статистику складу, Pydantic validation і HTTP status codes. Додано асинхронний HTTP-клієнт з Task, gather, Semaphore, timeout і retry, а також автоматизовані API-тести. Проєкт демонструє розділення API, сервісного та repository-рівнів і збереження даних у SQLite.